

Verification detail tabs — frontend integration guide

Recorded 2026-09-15. Covers the eleven endpoints behind a verification's review screen: `GET /v1/verifications/kyc/:workflowId/*` (five tabs) and `GET /v1/verifications/kyb/:workflowId/*` (six tabs). Companion to `verification-list-endpoints.md` (the list/queue view these tabs are drilled into from) and `api-integration-guide.md` (the full API reference).

Auth — read this before wiring anything up

Session (JWT) only. These routes are guarded by `PermissionsGuard({ allow: ['dashboard:access'] })` — no `CombinedAuthGuard`, no API key path at all. A request authenticated with a tenant API key gets a **401**, not a 403 — the guard doesn't recognize the credential as a session, full stop.

This is deliberate, not an oversight: a document's signed download URL and a director's identity detail are exactly the kind of customer-identifying material an API-key integration has no business pulling in bulk. If your frontend currently drives its other verification screens off an API key (the way the test console's "Verifications" list can), these eleven routes need your app's own signed-in session instead — same `Authorization: Bearer <token>` header, just a session token from `POST /auth/sign-in`, not `sk_...`.

A workflow requested through the wrong profile's path — a business workflow via `/verifications/kyc/...`, or an individual one via `/verifications/kyb/...` — 404s. The path doesn't just filter, it asserts.

Every route returns the usual envelope: `{ status, message, data }`. No `meta` — nothing here is paginated.

KYC — `GET /v1/verifications/kyc/:workflowId/*`

Five tabs: `documents`, `risk-score`, `aml-screening`, `device-information`, `liveness`.

documents

```
{
  "workflowId": "1fc4de4ab9024d7399016fb8c6fb48c9",
  "documents": [
    {
      "id": "241a587e4013477da99f7a69e592607e",
      "type": "selfie-photo",
      "category": "liveness-check",
      "status": "validated",
      "url": "https://...", // the bucket is private; this alone
doesn't fetch
      "signedUrl": "https://...&X-Amz-Signature=...", // time-limited (~1hr), use
this
      "contentType": "image/png",
      "idNumber": null,
      "issueDate": null,
    }
  ]
}
```

```

    "expiryDate": null,
    "customerId": "...",
    "customerType": "individual",
    "shareholderId": null,
    "extractedInformation": null // always null – no OCR capability
  },
  "riskScore": 0,
  "riskBand": "low", // null until risk-decisioning has scored the run
  "timeline": [ /* workflow events – see below */ ]
}

```

`signedUrl` is `null` for a document row with no file stored (`key` never set) — never render it as if it were an empty string. Each timeline entry is `{ id, action, actor, fromStatus, toStatus, detail, createdAt }` — the same shape as `events` on `GET /verifications/:workflowId`, not a new one.

`extractedInformation` is always `null`. The field exists on the shape so nothing breaks the day OCR extraction ships; it's not per-document absent, it's platform-wide absent today.

risk-score

```

{
  "available": true, // false, with risk: null, until risk-decisioning has
  scored the run
  "risk": {
    "runId": "...",
    "workflowId": "...",
    "score": 0, // 0-4
    "band": "low",
    "priority": "standard",
    "decision": "onboard", // onboard | review | block
    "canApprove": true, // false at/above the block threshold, or a PEP tier
    demanding escalation
    "requiresEscalation": false,
    "contributions": [
      { "code": "...", "factorSlug": "...", "message": "...", "points": 0, "source":
"verification", "evidence": {} }
    ],
    "warningThreshold": 2,
    "blockThreshold": 3,
    "decidedAt": "2026-09-14T22:46:40.295Z"
  }
}

```

No narrative recommendation sentence. `contributions` is the full structured input you'd compose one from — the mockups' "Business presents HIGH RISK with 3 contributing factors..." prose isn't generated server-side. If a screen needs that sentence, template it client-side from `contributions`, or treat it as a gap to flag rather than something to wait for silently.

aml-screening

```
{
  "tasks": [
    {
      "id": "...",
      "verificationType": "sanctions-screening",    // or pep-screening
      "outcome": "passed",                        // passed | failed | review-
required | pending | skipped | error
      "reasons": [
        { "code": "screening.clear", "severity": "info", "message": "No sanctions
match across 1 dataset(s)",
          "evidence": { "threshold": 70, "totalCandidates": 0, "datasetsSearched":
["sandbox_default"] } }
      ],
      "providerKey": "sandbox-aml",
      "providerReference": null,
      "attempts": 1,
      "completedAt": "2026-09-14T22:46:40.144Z"
    }
  ]
}
```

Only the sanctions-/pep-screening tasks — everything else on the run is excluded, this tab isn't a general task list. **No per-list breakdown** (OFAC / UN / EU / UK HMT rows) and no separate watchlist/warning sections — that's `reasons[]`'s `evidence.datasetsSearched`, not a mapped shape. As of the multi-provider change, a single task's `reasons[]` can already contain entries from more than one provider with no per-reason attribution — don't assume one reasons array means one provider call.

device-information

```
{
  "available": false,
  "ip": null,
  "geolocation": null,
  "device": null,
  "vpnDetected": null,
  "proxyDetected": null,
  "reason": "IP and device capture is not implemented (see TODO D10)"
}
```

Every field is always this — `available` is always `false` today. Nothing captures a submitting IP or device fingerprint anywhere in the platform; this isn't a per-workflow gap, render the whole tab as an empty state using `reason` as the message rather than polling or retrying.

liveness

```
{
  "available": true,                // false with task: null if there's no selfie-type
}
```

```

task on the run
  "livenessDetection": "skipped", // the task's outcome, verbatim – passed | failed
  | skipped | ...
  "matchScoreRate": null,          // 0-100 when a provider evidence carries a score,
  else null
  "providerKey": null,
  "providerReference": null,
  "attempts": 1,
  "completedAt": "2026-09-14T22:46:38.387Z",
  "reasons": [
    { "code": "verification.unsupported", "severity": "warning",
      "message": "\"selfie\" is not offerable in NG" }
  ]
}

```

Read this before building a real liveness screen. The dedicated liveness provider capability (matchScore, livenessPassed, a per-check breakdown of face detection / real person / no mask / eyes open / facing camera, completion time, attempt count) exists in provider-gateway but is **never actually invoked** — a selfie check runs through the same generic identity-lookup path as any other check today, confirmed live. For most countries there's no identity type mapped to "selfie", so the task settles `skipped` and this tab has nothing but that skip reason to show. `matchScoreRate` only ever gets a value on the rare path where the generic identity check *did* run and returned field-match scores — it is not a biometric match score. Don't build a liveness-detail UI expecting the rich mockup data (confidence %, checks-performed list, attempts, seconds) — none of it exists server-side yet.

KYB — GET /v1/verifications/kyb/:workflowId/*

Six tabs, a different set than KYC's — matched to the actual KYB review screens, not a renamed copy of the KYC tabs: `business-overview`, `risk-score`, `directors-officers`, `shareholders`, `document`, `compliance-checks`.

business-overview

```

{
  "workflowId": "...",
  "business": {
    "id": "...",
    "businessName": "TechVentures Nigeria Limited",
    "businessType": null,
    "countryCode": "NG",
    "address": { "city": "Lagos", "street": null, "country": "NG", "formatted":
"Lagos, NG ()" },
    "registrationDate": "2018-03-15",
    "industry": "technology",
    "contactEmail": "info@techventures.ng",
    "contactPhone": "+2348091234567",
    "website": null,
    "status": "review"
  },
  "registrationNumber": "RC-23456789", // best-effort, see below – null if never

```

```

submitted
  "tin": null, // same best-effort path, different document
type
  "activities": null, // never populated — not modeled anywhere
  "status": "in-review", // the *workflow's* status, distinct from
business.status above
  "riskScore": 0
}

```

registrationNumber and **tin** are not stored fields. `customer_business` has no registration-number or TIN column — both are read off whichever submitted document carries an `idNumber` for that type (`certificate-of-incorporation` → `registrationNumber` , `tin` → `tin`). If the tenant never uploaded that document, or uploaded it without an ID number, the field is `null` — this is not a bug, there's nowhere else for the platform to get it. `business.status` (the customer's own lifecycle status) and the top-level `status` (the *workflow's* status) are two different things that happen to both exist on this response — don't conflate them.

risk-score

Identical shape to the KYC tab above — same service method, just scoped to a business workflow.

directors-officers

```

{
  "directors": [
    {
      "id": "bf1733367b4d469e9c290e4f3bb9308f",
      "firstName": "Tejumade",
      "lastName": "Olomola",
      "sharePercentage": 60,
      "shareCountTotal": 60000,
      "type": "individual",
      "dateAppointed": "2018-03-15",
      "role": "Managing Director / CEO",
      "email": "tej@techventures.ng",
      "phone": "+2348052345678",
      "countryCode": "NG",
      "address": { "city": "Lagos", "country": "NG" },
      "nationality": null, // not captured on the shareholder record — always
null
      "idType": null, // same
      "idNumber": null, // same
      "screening": [
        { "verificationType": "sanctions-screening", "outcome": "passed", "reasons":
[ /* ... */ ] },
        { "verificationType": "pep-screening", "outcome": "passed", "reasons": [ /*
... */ ] }
      ]
    }
  ]
}

```

```
]
}
```

This is real, live data, not a stub — confirmed against a running run: the orchestrator already fans sanctions-/pep-screening out to every shareholder alongside the business itself (`verification_task.shareholderId` keys each attempt to the director it's about), so `screening[]` here is each director's own genuine AML result, not the business's. What's honestly absent: nationality and ID type/number were never captured on the shareholder record in the first place — this isn't a screening gap, it's a data-capture gap, `null` on every director regardless of what screening found.

shareholders

```
{
  "shareholders": [
    {
      "id": "...",
      "firstName": "Tejumade",
      "lastName": "Olomola",
      "sharePercentage": 60,
      "shareCountTotal": 60000,
      "type": "individual",
      "shareClass": null // ordinary/preference isn't captured – always null
    }
  ]
}
```

Same underlying shareholder records as `directors-officers`, without the screening detail — this is the share-capital-structure view, not a second copy of the AML result.

document

```
{
  "workflowId": "...",
  "documents": [
    {
      "id": "...",
      "type": "certificate-of-incorporation",
      "category": "certificate-of-incorporation",
      "status": "validated",
      "signedUrl": "https://...",
      "idNumber": "RC-23456789",
      "customerType": "business"
    }
  ]
}
```

Same signed-URL mechanism as KYC's `documents` tab, no `extractedInformation` field here (not part of this tab's shape). Two document types the KYB mockups show — Form 7 and Tax Clearance Certificate — **aren't in the platform's document-type enum at all**; a tenant literally cannot submit one today, so don't build UI expecting them to ever appear here.

compliance-checks

```
{
  "tasks": [
    {
      "id": "...",
      "verificationType": "sanctions-screening",
      "outcome": "passed",
      "reasons": [ /* ... */ ],
      "providerKey": "sandbox-aml"
    }
  ]
}
```

The business entity's own sanctions-/pep-screening tasks only — filtered to `shareholderId === null`. A director's screening result never appears here; it's exclusively on `directors-officers`. If a screen needs "every AML check on this run, business and directors together," it has to combine both tabs client-side — there's no combined endpoint.

Building the tab UI

Both controllers return every tab as a flat, independent `GET` — there's no "give me the whole detail page in one call." Fetch each tab lazily as the user opens it (the test console does exactly this, one request per tab click, cached for the life of the page) rather than firing all five or six requests on page load.

Gate which tab set you request on the workflow's own `profileType` — pull it from `GET /verifications/:workflowId` (or the list row) first, then hit `kyc/...` or `kyb/...` accordingly. Hitting the wrong path 404s.